

Copyright Notice

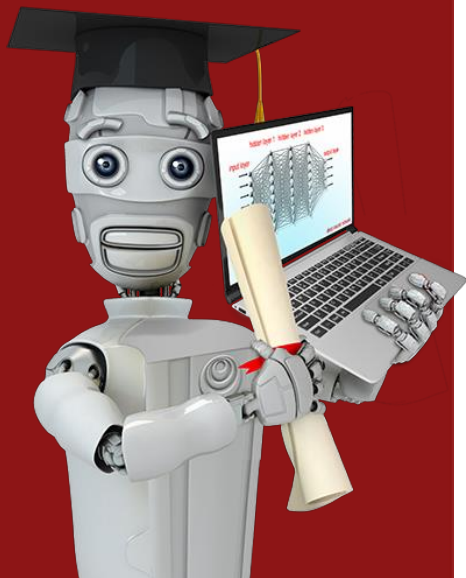
These slides are distributed under the Creative Commons License.

[DeepLearning.AI](#) makes these slides available for educational purposes. You may not use or distribute these slides for commercial purposes. You may make copies of these slides and use or distribute them for educational purposes as long as you cite [DeepLearning.AI](#) as the source of the slides.

For the rest of the details of the license, see <https://creativecommons.org/licenses/by-sa/2.0/legalcode>



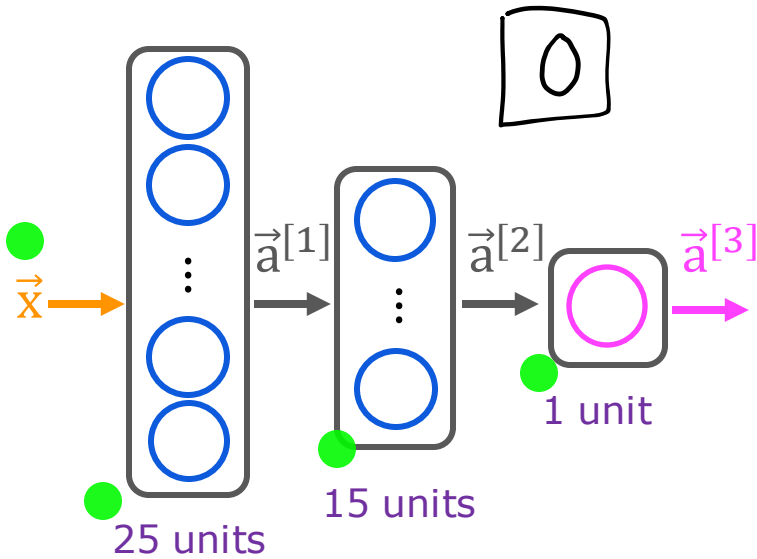
Stanford
ONLINE



Neural Network Training

TensorFlow implementation

Train a Neural Network in TensorFlow



Given set of (x, y) examples
How to build and train this in code?

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(units=25, activation='sigmoid')
    Dense(units=15, activation='sigmoid')
    Dense(units=1, activation='sigmoid')
])

from tensorflow.keras.losses import BinaryCrossentropy

model.compile(loss=BinaryCrossentropy())

model.fit(X, Y, epochs=100)
```

epochs: number of steps in gradient descent

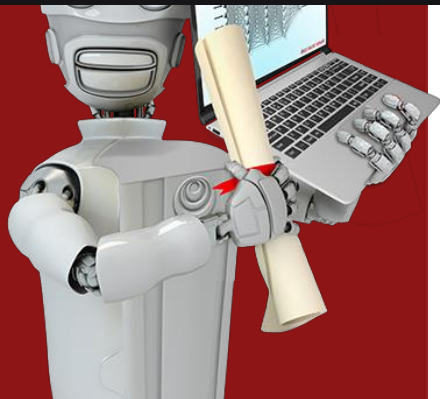
Neural Network Training

The Core Framework: The Three-Step Process

Both logistic regression and neural networks are trained using the same fundamental three-step process. Understanding this parallel is key to mastering the intuition.

1. **Specify the Model (Forward Propagation):** This defines how the output is calculated given an input x and the model's parameters (weights w and biases b).
2. **Define the Loss and Cost Function:** This defines *how* the model's performance will be measured.
3. **Minimize the Cost Function (Gradient Descent):** This is the optimization process that finds the best parameters to fit your data.

Let's dive deep into each step in the context of neural networks and TensorFlow.



Training Details

Model Training Steps TensorFlow

①

specify how to compute output given input x and parameters w, b (define model)

$$f_{\vec{w}, b}(\vec{x}) = ?$$

②

specify loss and cost

$$L(f_{\vec{w}, b}(\vec{x}), y) \quad \text{1 example}$$

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)})$$

③

Train on data to minimize $J(\vec{w}, b)$

logistic regression

```
z = np.dot(w, x) + b  
f_x = 1 / (1 + np.exp(-z))
```



logistic loss

```
loss = -y * np.log(f_x)  
       -(1-y) * np.log(1-f_x)
```

```
w = w - alpha * dj_dw  
b = b - alpha * dj_db
```

neural network

```
model = Sequential([  
    Dense(...)  
    Dense(...)  
    Dense(...)])
```

binary cross entropy

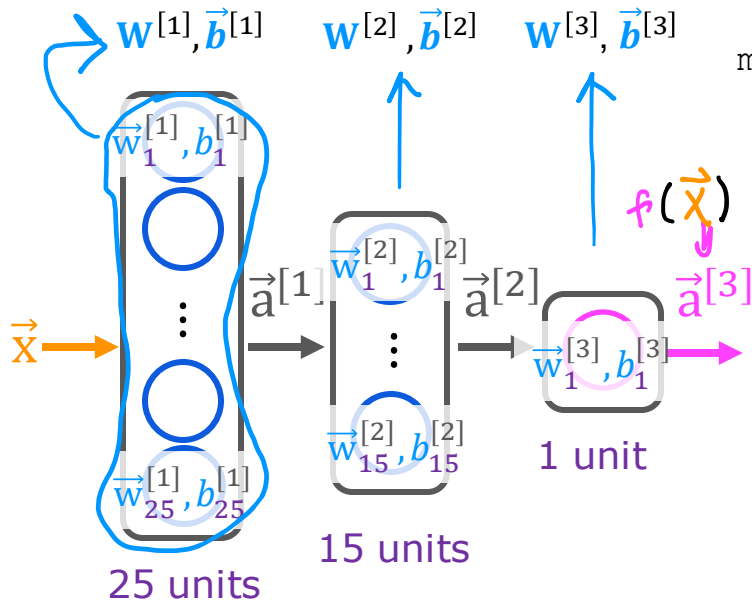
```
model.compile(  
    loss=BinaryCrossentropy())
```

```
model.fit(X, y, epochs=100)
```

1. Create the model

define the model

$$f(\vec{x}) = ?$$



```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential([
    Dense(units=25, activation='sigmoid')
    Dense(units=15, activation='sigmoid')
    Dense(units=1, activation='sigmoid')
])
```

Master's Insight #1: Why Columns and Not Rows?

Andrew explicitly stacks weights as **columns** in W . Why?
Because in linear algebra, the forward pass for a **batch** of data is:

$$Z = A_{in} \cdot W + b$$

- A_{in} shape: `(batch_size, num_inputs)`
- W shape: `(num_inputs, num_units)`
- Result Z shape: `(batch_size, num_units)`

By keeping W as `(inputs, units)`, we set up the matrix multiplication perfectly. This is exactly how TensorFlow's `Dense` layer stores its kernel (W is `(input_dim, units)`).

2. Loss and cost functions

Mnist digit classification problem  binary classification

$$L(f(\vec{x}), y) = -y \log(f(\vec{x})) - (1 - y) \log(1 - f(\vec{x}))$$

compare prediction vs. target

logistic loss

also known as binary cross entropy

`model.compile(loss= BinaryCrossentropy())`

regression

(predicting numbers and not categories)  mean squared error

`model.compile(loss= MeanSquaredError())`

$$J(\mathbf{W}, \mathbf{B}) = \frac{1}{m} \sum_{i=1}^m L(f(\vec{x}^{(i)}), y^{(i)})$$

$\mathbf{w}^{[1]}, \mathbf{w}^{[2]}, \mathbf{w}^{[3]}$ $\vec{b}^{[1]}, \vec{b}^{[2]}, \vec{b}^{[3]}$ $f_{\mathbf{W}, \mathbf{B}}(\vec{x})$

- **Loss Function (L)**: The error on a *single* training example (x, y) .
- **Cost Function (J)**: The average of the loss function over the *entire* training set.

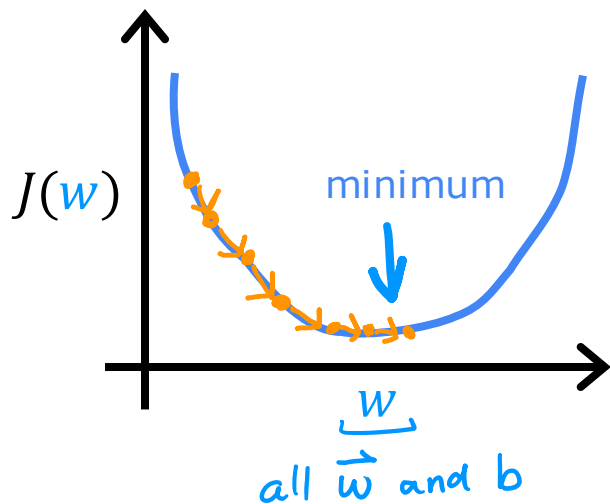
$$J(\mathbf{w}, \mathbf{b}) = (1/m) * \sum L(f(\mathbf{x}^{(i)}), y^{(i)})$$

```
from tensorflow.keras.losses import  
BinaryCrossentropy
```



```
from tensorflow.keras.losses import  
MeanSquaredError
```

3. Gradient descent



repeat {

$$w_j^{[l]} = w_j^{[l]} - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b_j^{[l]} = b_j^{[l]} - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

} Compute derivatives
for gradient descent
using "backpropagation"

2. Backpropagation: This is the key algorithm that TensorFlow uses to efficiently calculate those partial derivatives $(\partial J / \partial w)$ and $(\partial J / \partial b)$. The name is derived from the fact that it propagates the error *backwards* through the network.

- It starts at the output layer, calculates the error, and then moves backward, layer by layer, to compute the contribution of each weight and bias to the overall error.

```
model.fit(X, y, epochs=100)
```

1. Gradient Descent: It's an iterative algorithm that updates the parameters to move towards the minimum of the cost function.

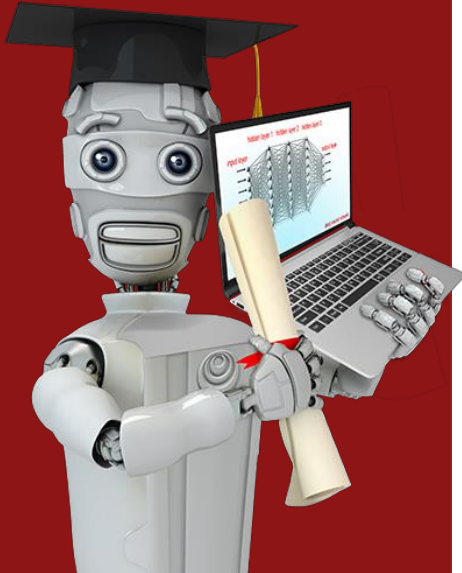
- $w = w - \alpha * (\partial J / \partial w)$
- $b = b - \alpha * (\partial J / \partial b)$
- Here, α is the learning rate.

Neural network libraries

Use code libraries instead of coding "from scratch"



Good to understand the implementation
(for tuning and debugging).



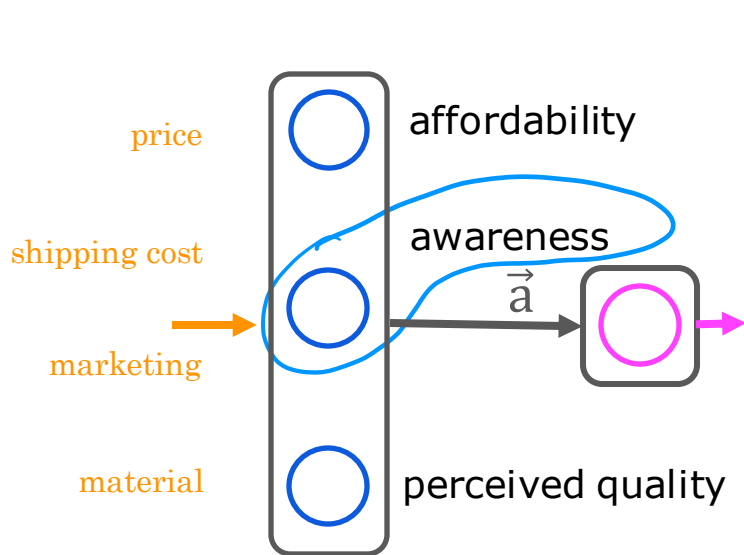
A Note on Activation Functions

The lectures mention that replacing the sigmoid activation function can make your neural network work much better. This is a crucial point for future weeks. Sigmoid is great for the output layer in binary classification (because it outputs a value between 0 and 1), but for hidden layers, it can cause problems like the "vanishing gradient" problem. **Rectified Linear Unit (ReLU)** is often a much better choice for hidden layers because it's faster and avoids this issue.

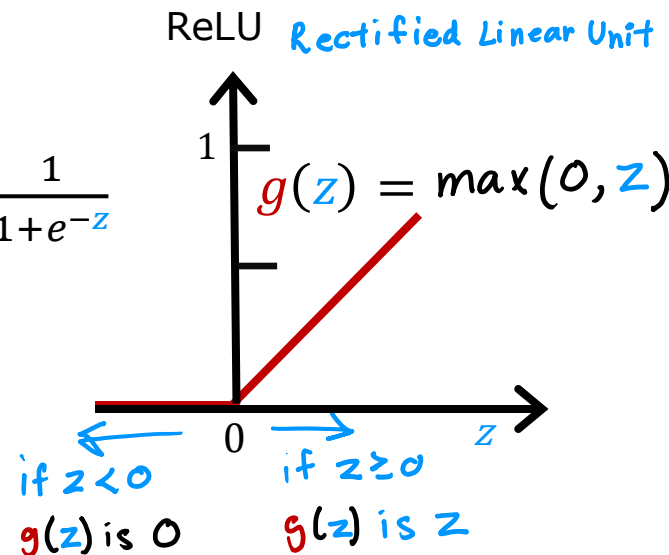
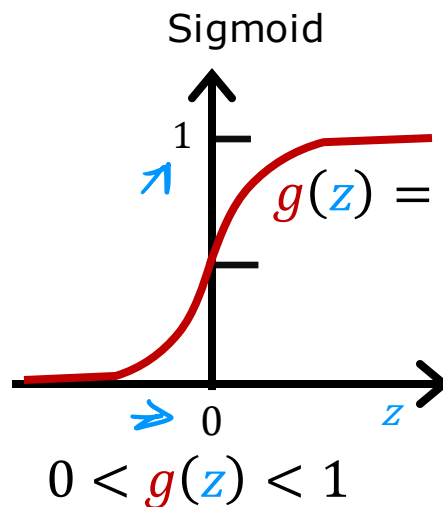
Activation Functions

Alternatives to the sigmoid activation

Demand Prediction Example



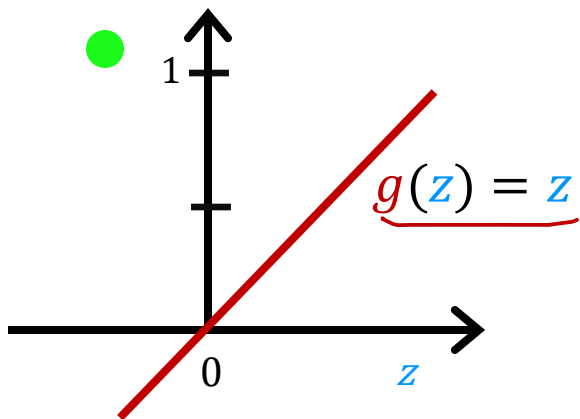
$$a_2^{[1]} = g(\overbrace{\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]}}^z)$$



Examples of Activation Functions

"No activation function"

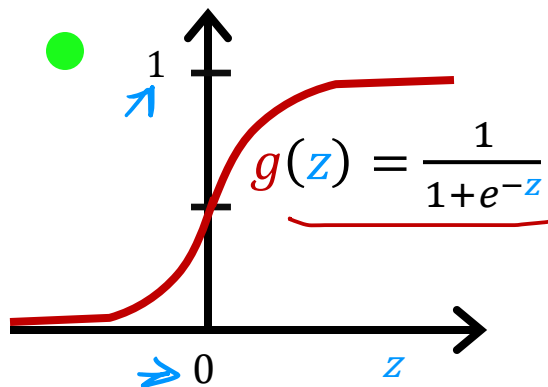
Linear activation function



$$a = g(z) = \underbrace{\vec{w} \cdot \vec{x} + b}_z$$

$$a_2^{[1]} = g(\overbrace{\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]}}^z)$$

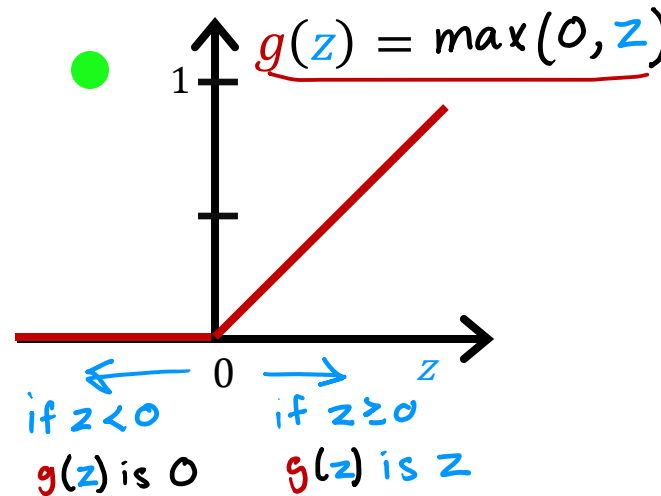
Sigmoid

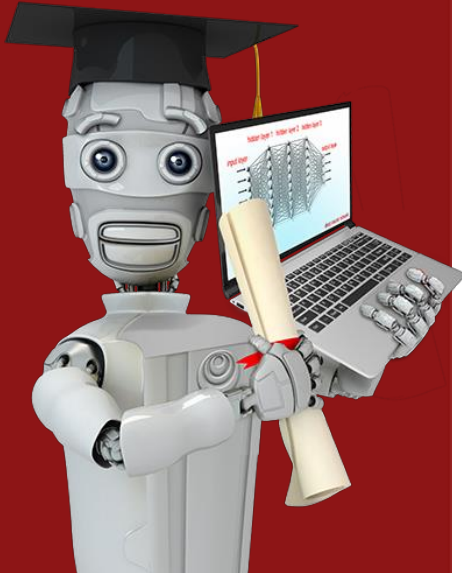


$$0 < g(z) < 1$$

Later: softmax activation

ReLU Rectified Linear Unit

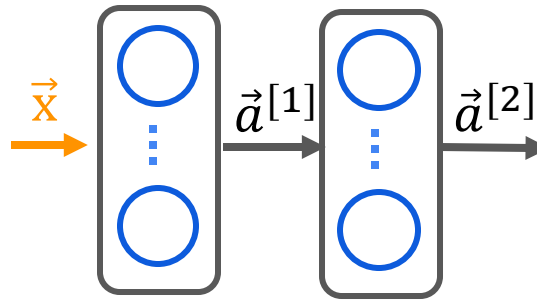




Activation Functions

Choosing activation functions

Output Layer

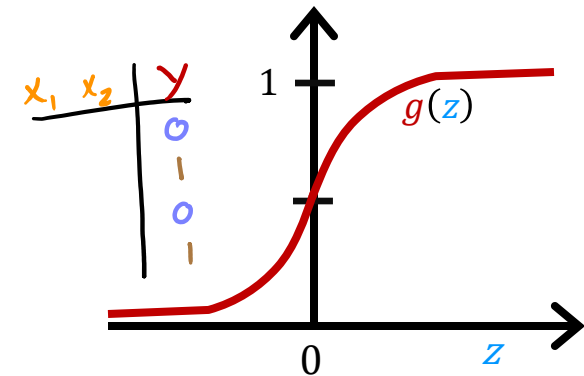


$$\vec{a}^{[3]} = f(\vec{x})$$

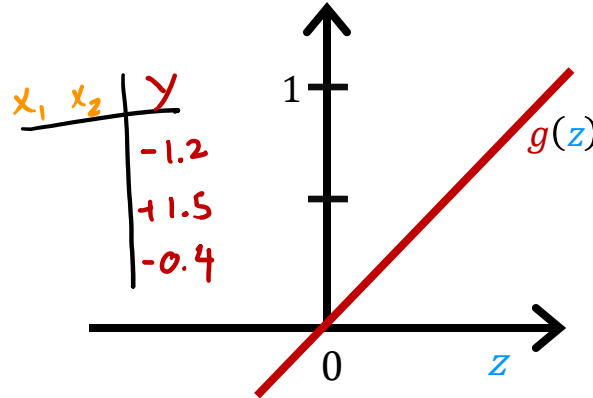
$$f(\vec{x}) = a_1^{[3]} = g(z_1^{[3]})$$

Choosing $g(z)$ for output layer?

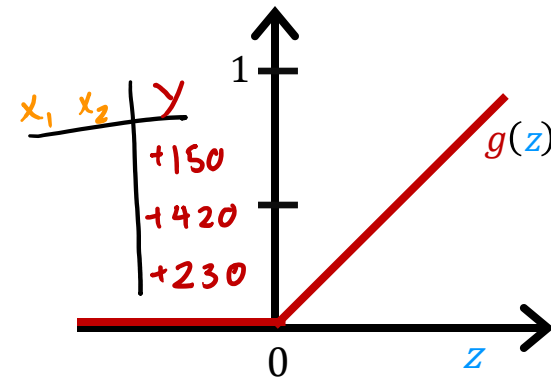
- Binary classification
- Sigmoid
- $y=0/1$



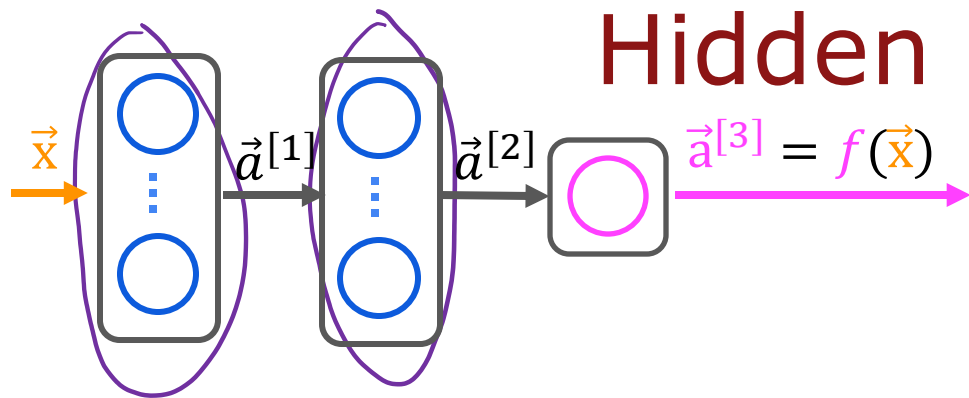
- Regression
- Linear activation function
- $y = +/-$



- Regression
- ReLU
- $Y = 0$ or $+$

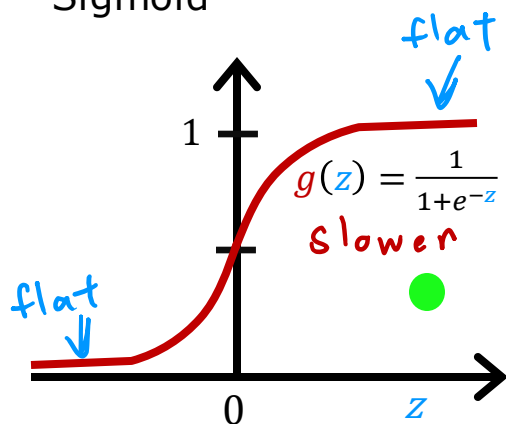


Hidden Layer

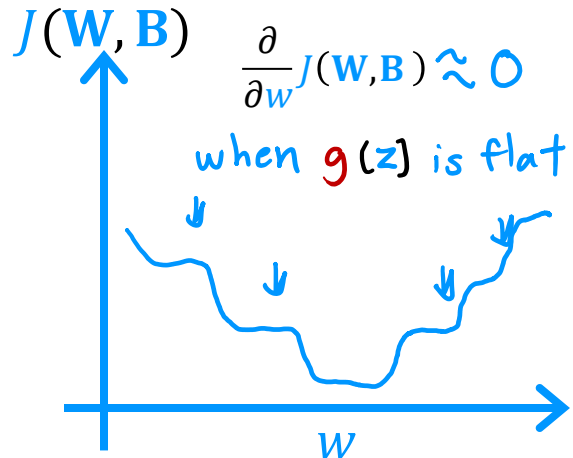


Choosing $g(z)$ for hidden layer

Sigmoid



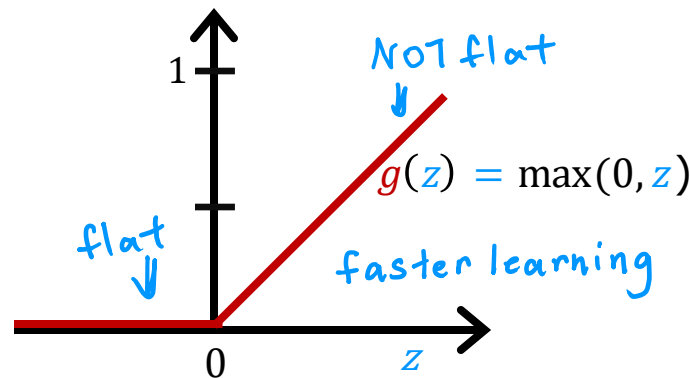
$J(W, B)$



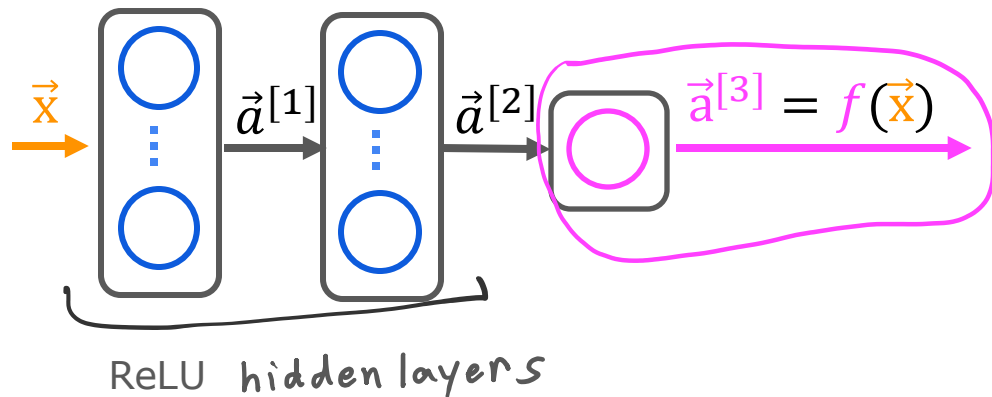
most common choice

ReLU

faster



Choosing Activation Summary



binary classification

activation='sigmoid'

regression y negative / positive

activation='linear'

regression $y \geq 0$

activation='relu'

```
from tf.keras.layers import Dense
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=1, activation='sigmoid')
])
```

layer1
layer2
layer3

or 'linear'
or 'relu'

2. The Evolutionary Timeline of Activations

To fix the mess that ReLU left behind, researchers created the next generations of activation functions that you brought up earlier:

Sigmoid / Tanh

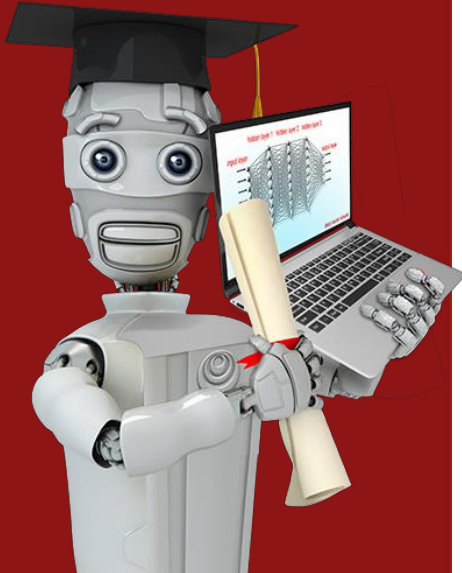
▼ (Fixes Vanishing Gradient)

ReLU (But introduces Dying ReLU)

└─ Leaky ReLU / PReLU (Fixes Dying ReLU with a small slope like 0.01x)

▼ (Fixes "sharp corners" for massive models)

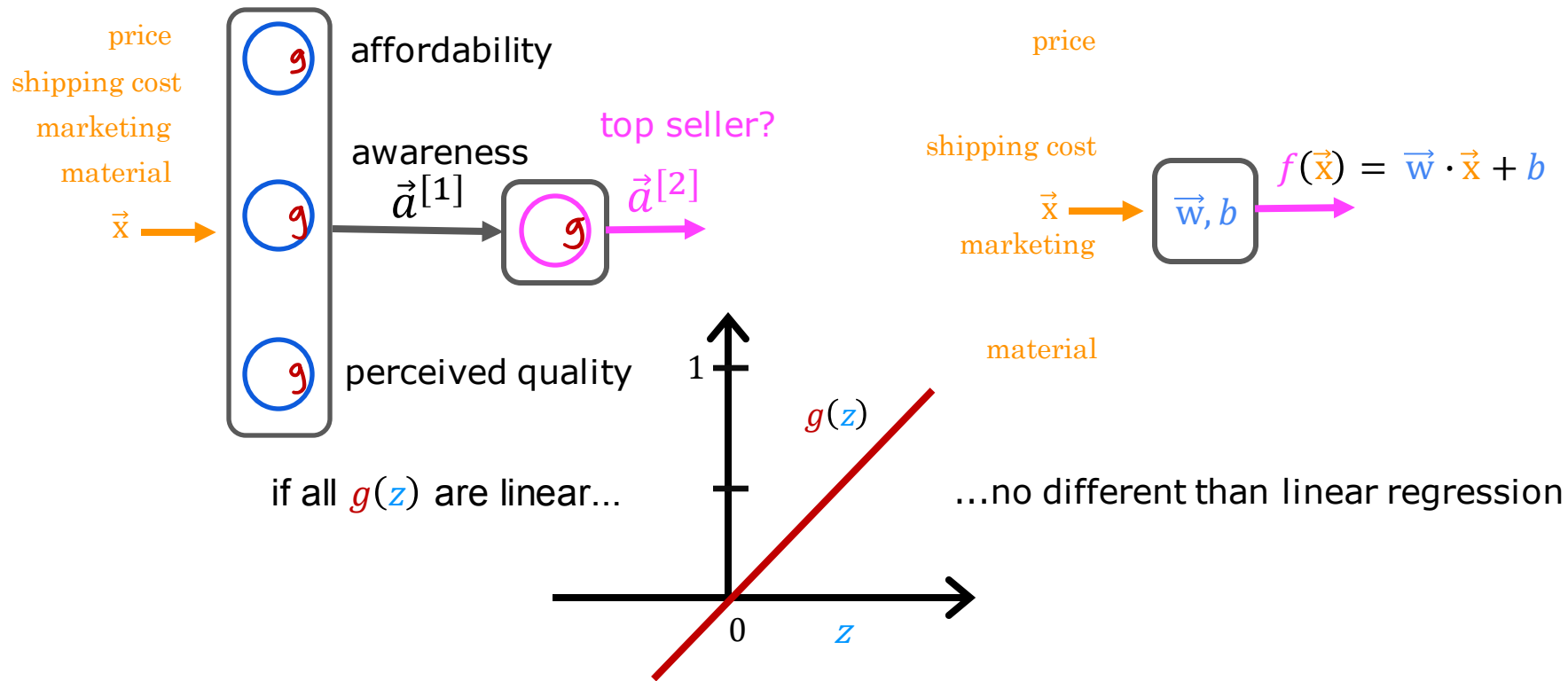
GELU / SwiGLU (Smooth, gated curves used in modern LLMs)



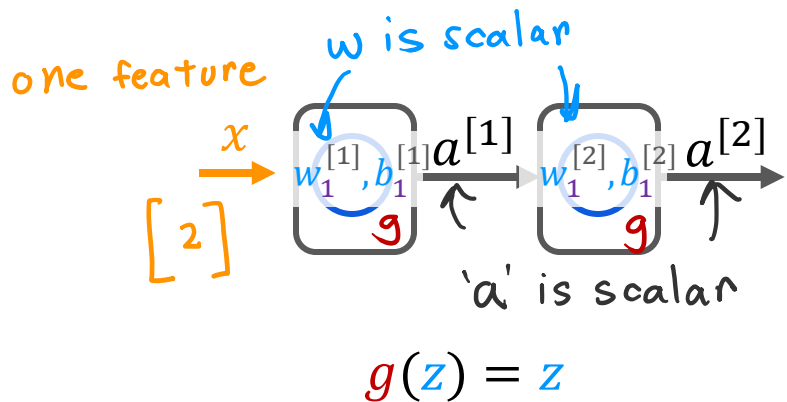
Activation Functions

Why do we need
activation functions?

Why do we need activation functions?



Linear Example



$$a^{[1]} = w_1^{[1]} x + b_1^{[1]}$$

$$a^{[2]} = w_1^{[2]} a^{[1]} + b_1^{[2]}$$

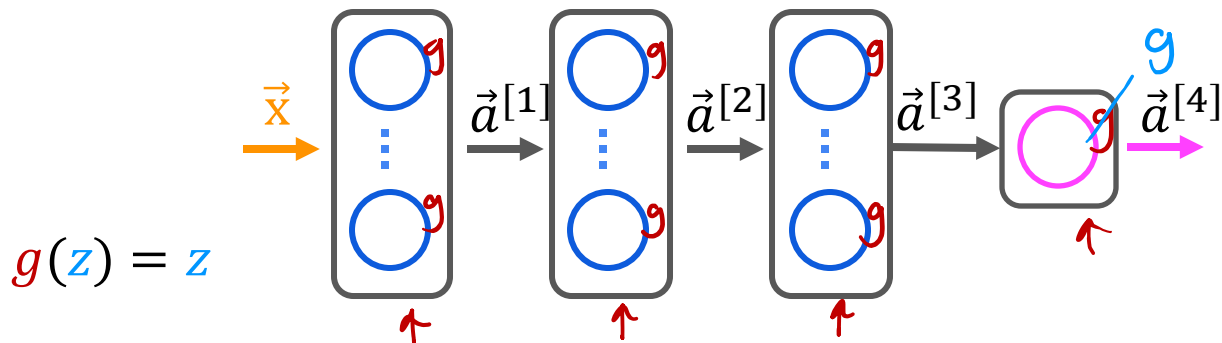
$$= w_1^{[2]} (w_1^{[1]} x + b_1^{[1]}) + b_1^{[2]}$$

$$\vec{a}^{[2]} = (\underbrace{\vec{w}_1^{[2]} \vec{w}_1^{[1]}}_w) x + \underbrace{w_1^{[2]} b_1^{[1]} + b_1^{[2]}}_b$$

$$\vec{a}^{[2]} = w x + b$$

$$f(x) = wx + b \quad \text{linear regression}$$

Example



$$g(z) = z$$

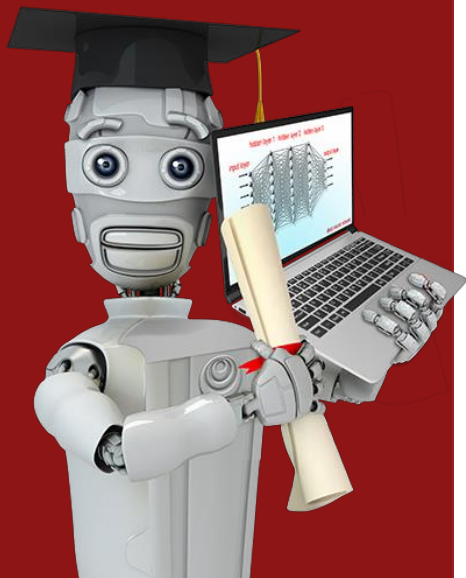
$$\vec{a}^{[4]} = \vec{w}_1^{[4]} \cdot \vec{a}^{[3]} + b_1^{[4]}$$

all linear (including output)
↳ equivalent to linear regression

$$\vec{a}^{[4]} = \frac{1}{1 + e^{-(\vec{w}_1^{[4]} \cdot \vec{a}^{[3]} + b_1^{[4]})}}$$

output activation is sigmoid
(hidden layers still linear)
↳ equivalent to logistic regression

Don't use linear activations in hidden layers (use ReLU)



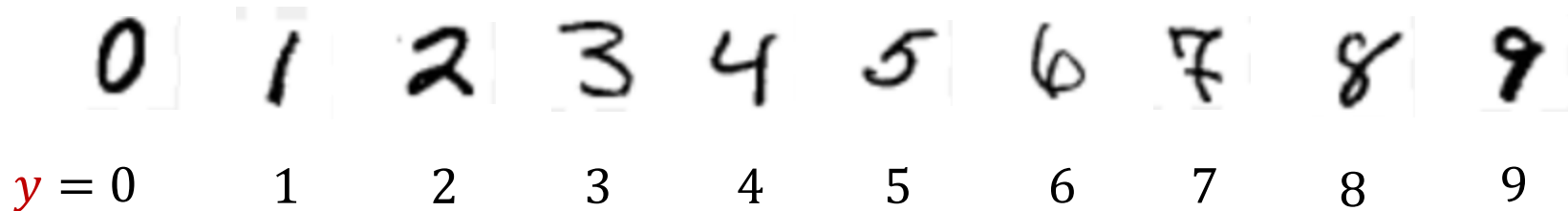
Multiclass Classification

Multiclass

1. The Problem Definition (Video 60)

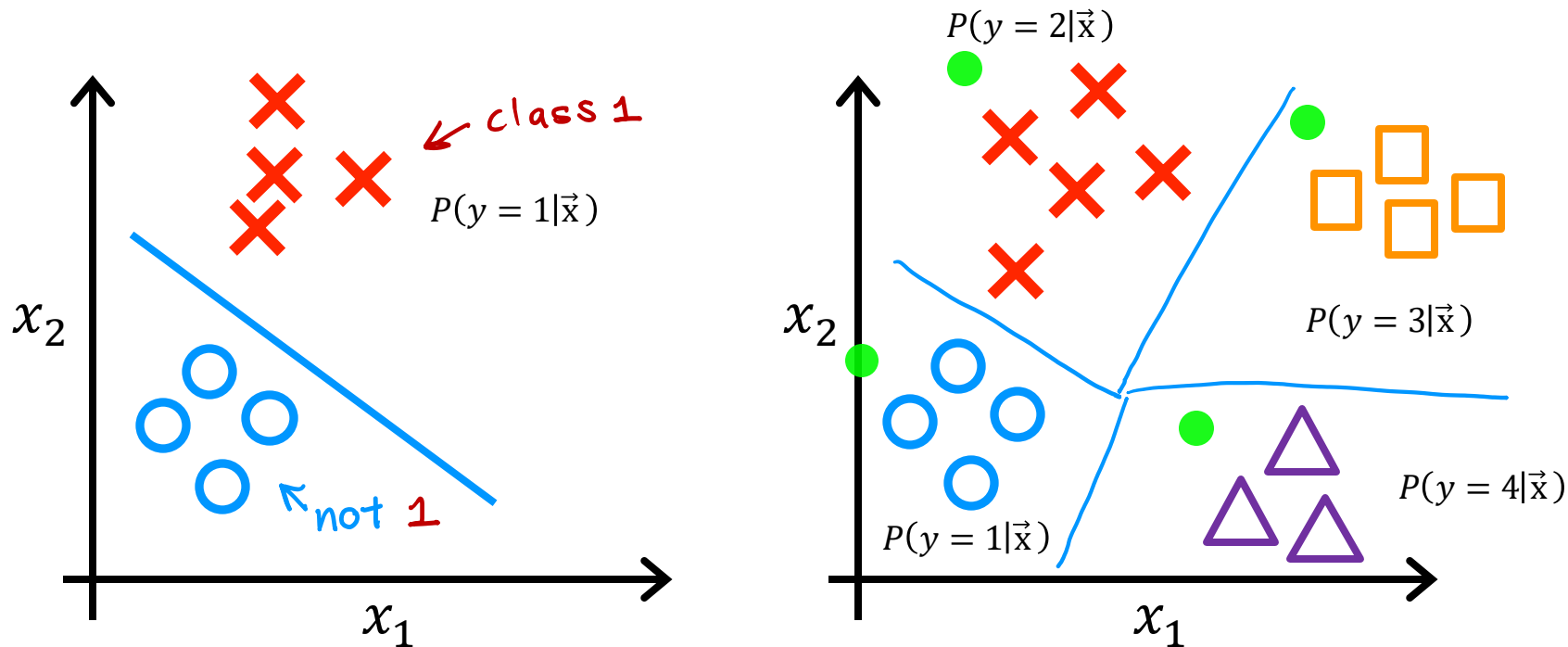
- **Binary vs. Multi-class:** In binary, y is 0 or 1. In multi-class, y can take more than 2 discrete values (e.g., digits 0-9, 4 types of diseases, or 4 types of product defects).
- **Decision Boundary:** Instead of a single line separating 2 regions, the algorithm learns boundaries that partition the feature space into **N distinct regions** (e.g., 4 regions for 4 classes).

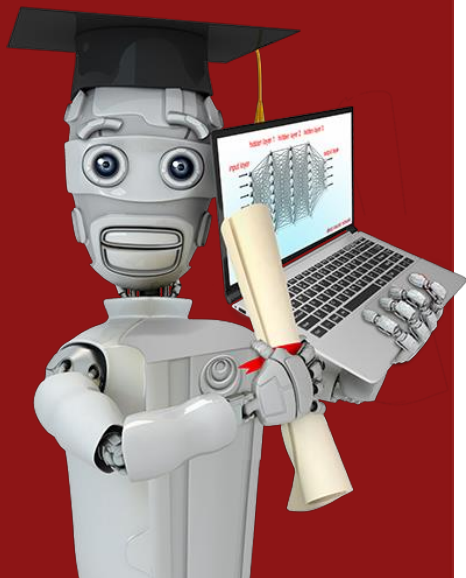
MNIST example



multiclass classification problem:
target y can take on more than two possible values

Multiclass classification example





Multiclass Classification

Softmax

- Review: Logistic regression computes $z = w \cdot x + b$, then $a1 = \text{sigmoid}(z) = P(y=1)$. We also implicitly compute $a2 = 1 - a1 = P(y=0)$.

Logistic regression
(2 possible output values)

$$z = \vec{w} \cdot \vec{x} + b$$

$$\times a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x}) \quad 0.71$$

$$\circ a_2 = 1 - a_1 = P(y=0|\vec{x}) \quad 0.29$$

Softmax regression
(N possible outputs) $y=1,2,3,\dots,N$

$$z_j = \vec{w}_j \cdot \vec{x} + b_j \quad j = 1, \dots, N$$

parameters $w_1, w_2, \dots, w_N$
 $b_1, b_2, \dots, b_N$

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}} = P(y=j|\vec{x})$$

$$\text{note: } a_1 + a_2 + \dots + a_N = 1$$

Softmax regression (4 possible outputs) $y=1,2,3,4$

$$\times z_1 = \vec{w}_1 \cdot \vec{x} + b_1$$

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=1|\vec{x}) \quad 0.30$$

$$\circ z_2 = \vec{w}_2 \cdot \vec{x} + b_2$$

$$a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=2|\vec{x}) \quad 0.20$$

$$\square z_3 = \vec{w}_3 \cdot \vec{x} + b_3$$

$$a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=3|\vec{x}) \quad 0.15$$

$$\triangle z_4 = \vec{w}_4 \cdot \vec{x} + b_4$$

$$a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=4|\vec{x}) \quad 0.35$$

Cost

2. The Math: Generalizing Logistic Regression (Video 61)

- **Review:** Logistic regression computes $z = w \cdot x + b$, then $a_1 = \text{sigmoid}(z) = P(y=1)$. We also implicitly compute $a_2 = 1 - a_1 = P(y=0)$.
- **The Expansion (Softmax):** For N classes, we compute N separate linear functions:
 - $z_1 = w_1 \cdot x + b_1$
 - $z_2 = w_2 \cdot x + b_2$
 - ...
 - $z_N = w_N \cdot x + b_N$
- **Softmax Formula:** To turn these into probabilities, we apply:
 - $a_j = e^{(z_j)} / (e^{(z_1)} + e^{(z_2)} + \dots + e^{(z_N)})$
- **Interpretation:** a_j = The model's estimated probability that $y = j$, given the input features x .
- **Key Property:** By design, $a_1 + a_2 + \dots + a_N = 1.0$ (they always sum to exactly 1).

3. The Cost Function (Loss) for Softmax (Video 61)

- Andrew emphasizes we do not use squared error. We use the **Categorical Cross-Entropy Loss**.
- **The Rule:** If the true label for a training example is class j (e.g., $y = j$), the loss for that single example is:
 - **Loss** = $-\log(a_j)$
- **Intuition:**
 - If the model is highly confident and correct (a_j close to 1.0) $\rightarrow -\log(1.0)$ is near 0 (Low Loss).
 - If the model is uncertain or wrong (a_j is small, e.g., 0.1) $\rightarrow -\log(0.1)$ is large (High Loss).
- **Overall Cost:** The total cost is the **average** of this loss over *all* training examples.

4. Connection to Neural Networks (Video 61)

- To use this in a Neural Network:
 - The **Output Layer** must have N units (one for each class).
 - Compute the raw logits z (using a linear activation).
 - The loss function handles the Softmax internally.

Cost

2. The Math: Generalizing Logistic Regression (Video 61)

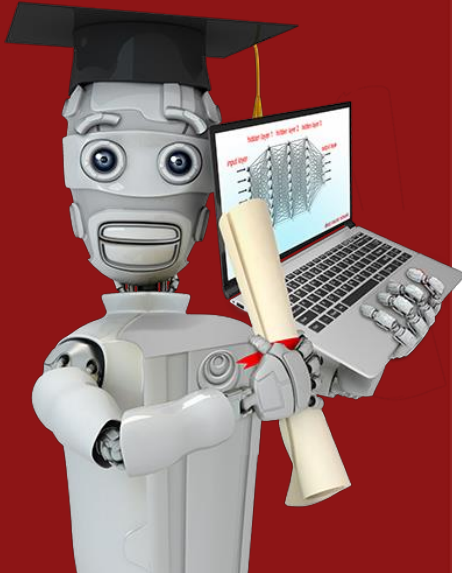
- **Review:** Logistic regression computes $z = w \cdot x + b$, then $a_1 = \text{sigmoid}(z) = P(y=1)$. We also implicitly compute $a_2 = 1 - a_1 = P(y=0)$.
- **The Expansion (Softmax):** For N classes, we compute N separate linear functions:
 - $z_1 = w_1 \cdot x + b_1$
 - $z_2 = w_2 \cdot x + b_2$
 - ...
 - $z_N = w_N \cdot x + b_N$
- **Softmax Formula:** To turn these into probabilities, we apply:
 - $a_j = e^{(z_j)} / (e^{(z_1)} + e^{(z_2)} + \dots + e^{(z_N)})$
- **Interpretation:** a_j = The model's estimated probability that $y = j$, given the input features x .
- **Key Property:** By design, $a_1 + a_2 + \dots + a_N = 1.0$ (they always sum to exactly 1).

3. The Cost Function (Loss) for Softmax (Video 61)

- Andrew emphasizes we do not use squared error. We use the **Categorical Cross-Entropy Loss**.
- **The Rule:** If the true label for a training example is class j (e.g., $y = j$), the loss for that single example is:
 - **Loss** = $-\log(a_j)$
- **Intuition:**
 - If the model is highly confident and correct (a_j close to 1.0) $\rightarrow -\log(1.0)$ is near 0 (Low Loss).
 - If the model is uncertain or wrong (a_j is small, e.g., 0.1) $\rightarrow -\log(0.1)$ is large (High Loss).
- **Overall Cost:** The total cost is the **average** of this loss over *all* training examples.

4. Connection to Neural Networks (Video 61)

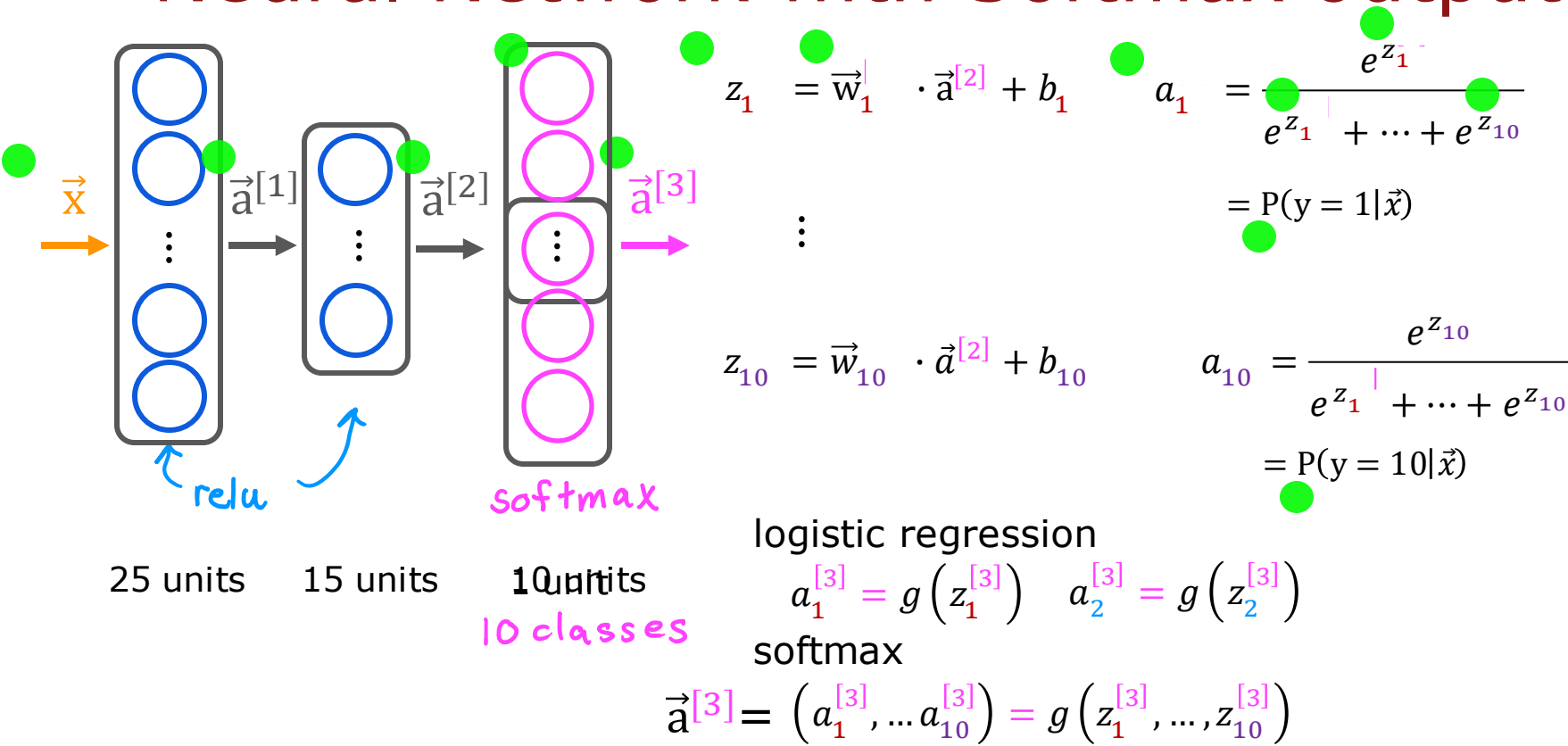
- To use this in a Neural Network:
 - The **Output Layer** must have N units (one for each class).
 - Compute the raw logits z (using a linear activation).
 - The loss function handles the Softmax internally.



Multiclass Classification

Neural Network with
Softmax output

Neural Network with Softmax output



MNIST with softmax

① specify the model

$$f_{\vec{w},b}(\vec{x}) = ?$$

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    • Dense(units=25, activation='relu')
    • Dense(units=15, activation='relu')
    • Dense(units=10, activation='softmax')
])
```

② specify loss and cost

$$L(f_{\vec{w},b}(\vec{x}), y)$$

```
from tensorflow.keras.losses import
    SparseCategoricalCrossentropy

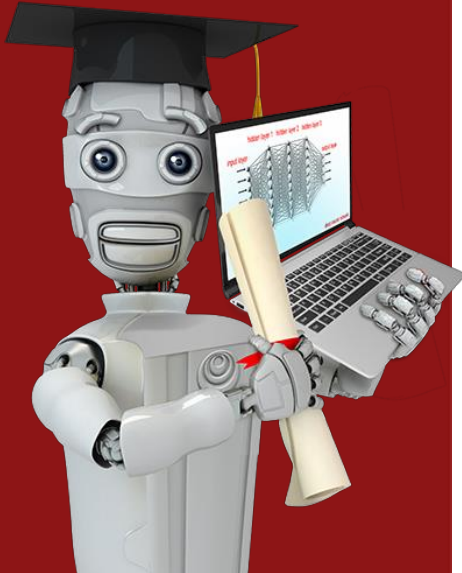
model.compile(loss= SparseCategoricalCrossentropy() )

model.fit(X,Y,epochs=100)
```

③ Train on data to minimize $J(\vec{w},b)$

Note: better (recommended) version later.

Don't use the version shown here!



Multiclass Classification

Improved implementation
of softmax

Numerical Roundoff Errors

option 1

$$x = \frac{2}{10,000}$$

option 2

$$x = \left(1 + \frac{1}{10,000}\right) - \left(1 - \frac{1}{10,000}\right) =$$


Numerical Roundoff Errors

More numerically accurate implementation of logistic loss:

$$1 + \frac{1}{10,000} \quad 1 - \frac{1}{10,000}$$

Logistic regression:

$$\boxed{a} = g(z) = \frac{1}{1 + e^{-z}}$$

```
model = Sequential([  
    Dense(units=25, activation='relu')  
    Dense(units=15, activation='relu') 'linear'  
    Dense(units=10, activation='sigmoid')
```

Original loss

$$\text{loss} = -y \log(\boxed{a}) - (1 - y) \log(1 - \boxed{a})$$

```
model.compile(loss=BinaryCrossEntropy() )  
model.compile(loss=BinaryCrossEntropy(from_logits=True) )
```

More accurate loss (in code)

$$\text{loss} = -y \log\left(\frac{1}{1 + e^{-z}}\right) - (1 - y) \log\left(1 - \frac{1}{1 + e^{-z}}\right)$$

logit: z

More numerically accurate implementation of softmax

Softmax regression

$$(a_1, \dots, a_{10}) = g(z_1, \dots, z_{10})$$

$$\text{Loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ \vdots \\ -\log a_{10} & \text{if } y = 10 \end{cases}$$

```
model = Sequential([
    Dense(units=25, activation='relu')
    Dense(units=15, activation='relu')
    Dense(units=10, activation='softmax')
```

'linear'

```
model.compile(loss=SparseCategoricalCrossEntropy())
```

More Accurate

$$L(\vec{a}, y) = \begin{cases} -\log \frac{e^{z_1}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 1 \\ \vdots \\ -\log \frac{e^{z_{10}}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 10 \end{cases}$$

```
model.compile(loss=SparseCrossEntropy(from_logits=True))
```

MNIST (more numerically accurate)

```
model    import tensorflow as tf
         from tensorflow.keras import Sequential
         from tensorflow.keras.layers import Dense
         model = Sequential([
             Dense(units=25, activation='relu')
             Dense(units=15, activation='relu')
             Dense(units=10, activation='linear') ])

loss     from tensorflow.keras.losses import
         SparseCategoricalCrossentropy

         model.compile(..., loss=SparseCategoricalCrossentropy(from_logits=True) )

fit      model.fit(X,Y,epochs=100)

predict  logits = model(X) ← not  $a_1 \dots a_{10}$ 
         f_x = tf.nn.softmax(logits)           is  $z_1 \dots z_{10}$ 
```

logistic regression (more numerically accurate)

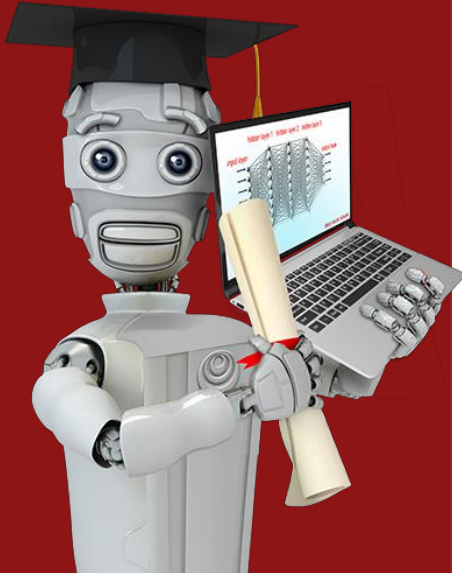
```
model    model = Sequential([
          Dense(units=25, activation='sigmoid')
          Dense(units=15, activation='sigmoid')
          Dense(units=1, activation='linear')
          from tensorflow.keras.losses import
            BinaryCrossentropy

loss      model.compile(..., BinaryCrossentropy(from_logits=True)) )

          model.fit(X,Y,epochs=100)

fit        logit = model(X)       $z$  ↘

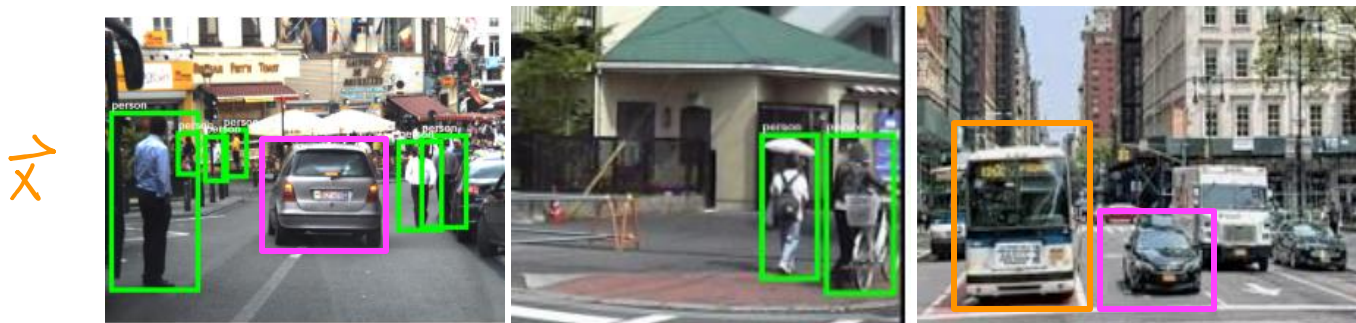
predict    f_x = tf.nn.sigmoid(logit)
```



Multi-label Classification

Classification with
multiple outputs
(Optional)

Multi-label Classification



Is there a car?

yes
no
yes $y = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$

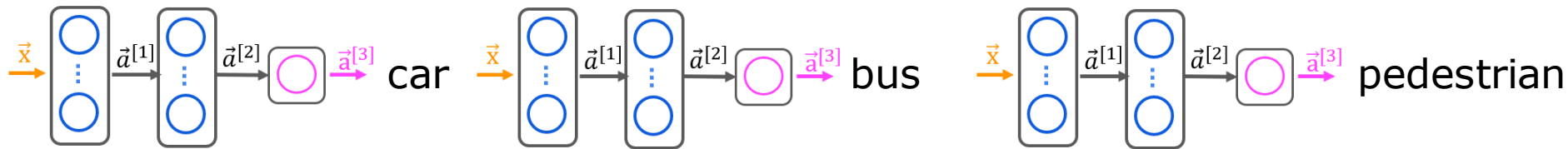
Is there a bus?

no
no
yes $y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$

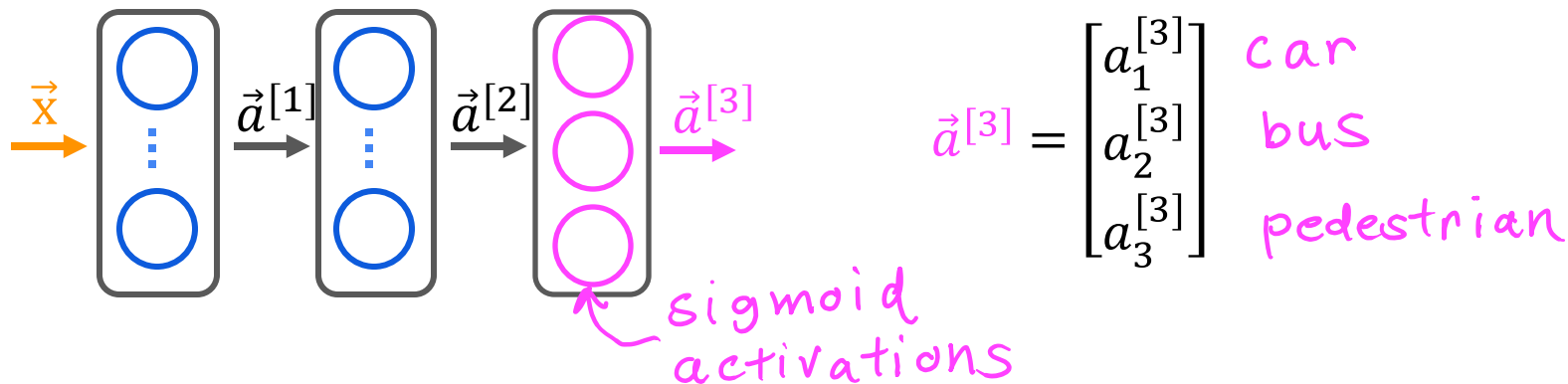
Is there a pedestrian

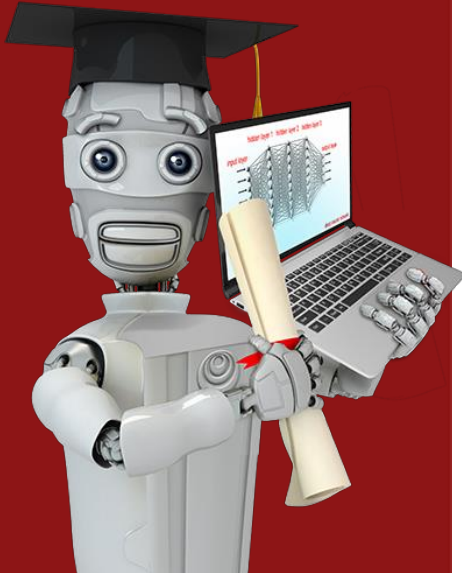
yes
yes
no $y = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$

Multiple classes



Alternatively, train one neural network with three outputs





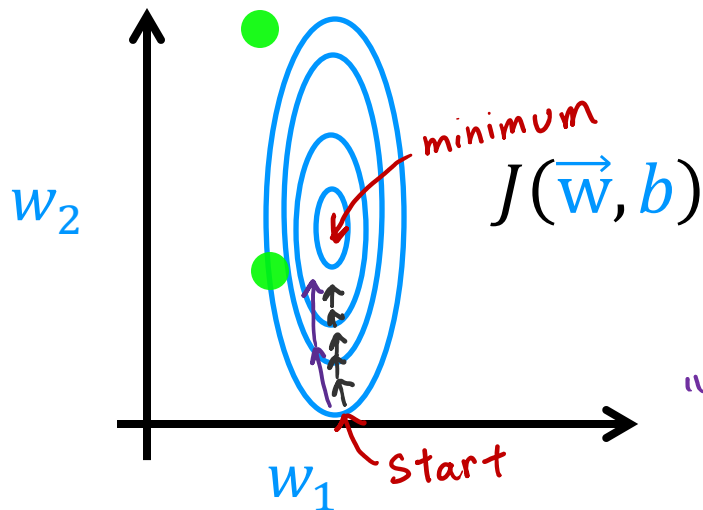
Additional Neural Network Concepts

Advanced Optimization

Gradient Descent

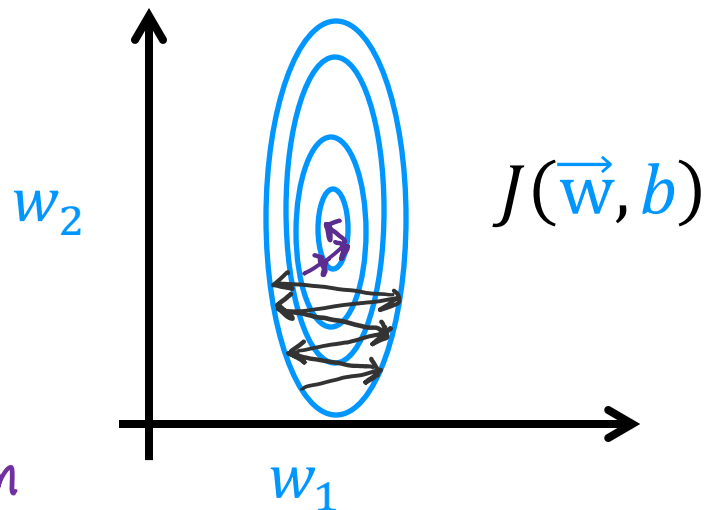
$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

α learning rate



Go faster – increase α

"Adam" algorithm



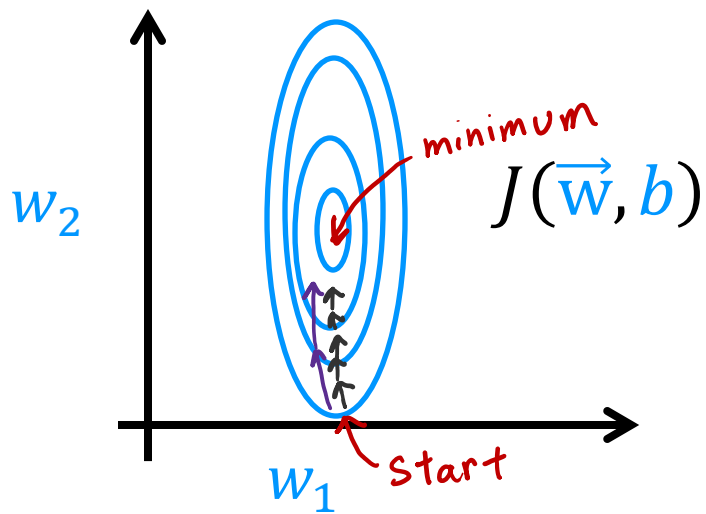
Go slower – decrease α

Adam Algorithm Intuition

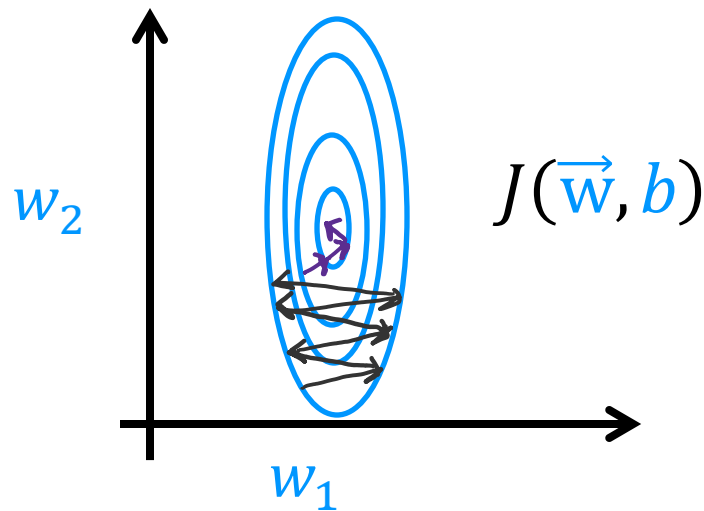
Adam: Adaptive Moment estimation *not just one α*

$$\begin{aligned}w_1 &= w_1 - \underbrace{\alpha_1}_{\text{adaptive}} \frac{\partial}{\partial w_1} J(\vec{w}, b) \\&\vdots \\w_{10} &= w_{10} - \underbrace{\alpha_{10}}_{\text{adaptive}} \frac{\partial}{\partial w_{10}} J(\vec{w}, b) \\b &= b - \underbrace{\alpha_{11}}_{\text{adaptive}} \frac{\partial}{\partial b} J(\vec{w}, b)\end{aligned}$$

Adam Algorithm Intuition



If w_j (or b) keeps moving in same direction, increase α_j .



If w_j (or b) keeps oscillating, reduce α_j .

MNIST Adam

model

```
model = Sequential([  
    tf.keras.layers.Dense(units=25, activation='sigmoid')  
    tf.keras.layers.Dense(units=15, activation='sigmoid')  
    tf.keras.layers.Dense(units=10, activation='linear')  
])
```

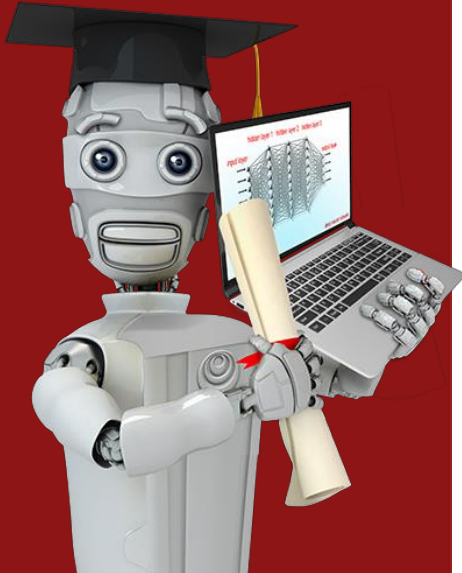
compile

$$\alpha = 10^{-3} = 0.001$$

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),  
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

fit

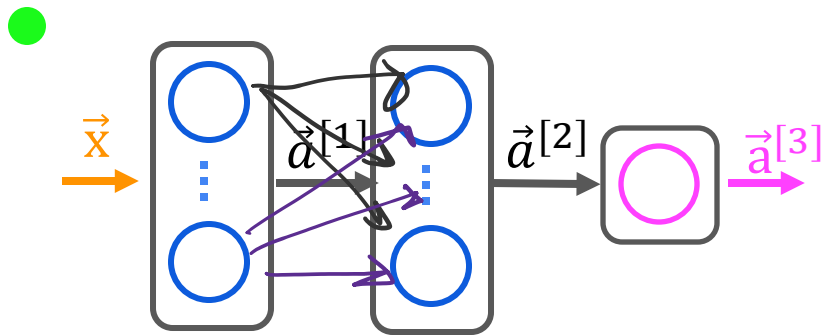
```
model.fit(X, Y, epochs=100)
```



Additional Neural Network Concepts

Additional Layer Types

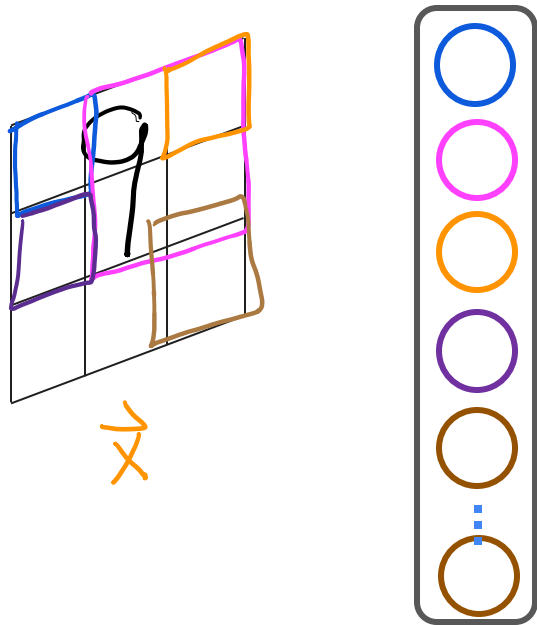
Dense Layer



Each neuron output is a function of all the activation outputs of the previous layer.

$$\vec{a}_1^{[2]} = g\left(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]}\right)$$

Convolutional Layer



Each Neuron only looks at part of the previous layer's inputs.

Why?

- Faster computation
- Need less training data (less prone to overfitting)

Convolutional Neural Network

